Trajectory Planning

Graduate Course INTR-6000P

Week 10 - Lecture 19

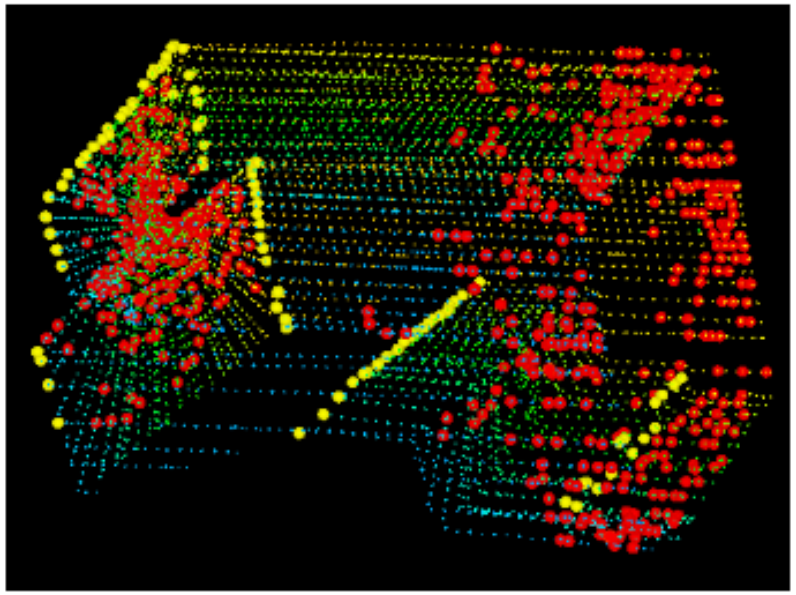
Changhao Chen

Assistant Professor

HKUST (GZ)

Recap: Point Cloud Registration

- Objective: **find rigid transform T ∈ SE(3) aligning point sets**
- Error metric: point-to-point vs point-to-plane
- Non-linear least squares (Gauss-Newton, ICP variants)



Align two point clouds → estimate relative motion (SE(3))

$$P_j = RP_i + t$$

Given:

- Source cloud: $\mathbf{P} = \{p_i\}$
- Target cloud: $\mathbf{Q} = \{q_j\}$

Find transform $\mathbf{T} = [R, t] \in SE(3)$

$$\mathbf{T}^* = \arg\min_{R,t} \sum_i \|Rp_i + t - q_{\pi(i)}\|^2$$

Where $\pi(i)$ is correspondence index.

Recap: ICP Objective

a) Point-to-Point ICP cost

$$E = \sum_i \| Rp_i + t - q_{\pi(i)} \|^2$$

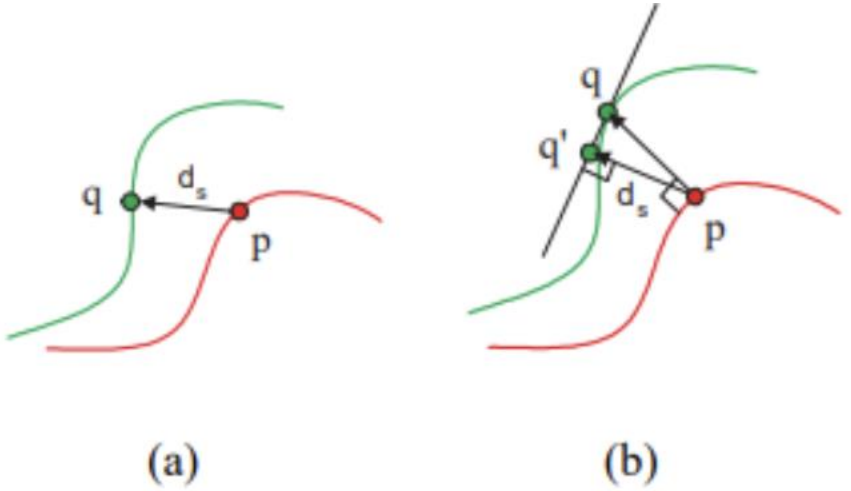
Minimizes the Euclidean distance between corresponding **points**.

b) Point-to-Plane ICP cost

$$E = \sum_i ((Rp_i + t - q_{\pi(i)}) \cdot n_j)^2$$

Minimizes the distance from the transformed source point to the **tangent plane** of the corresponding target point.

where n_j is target point normal.
Point-to-plane better for LiDAR because surfaces dominate.

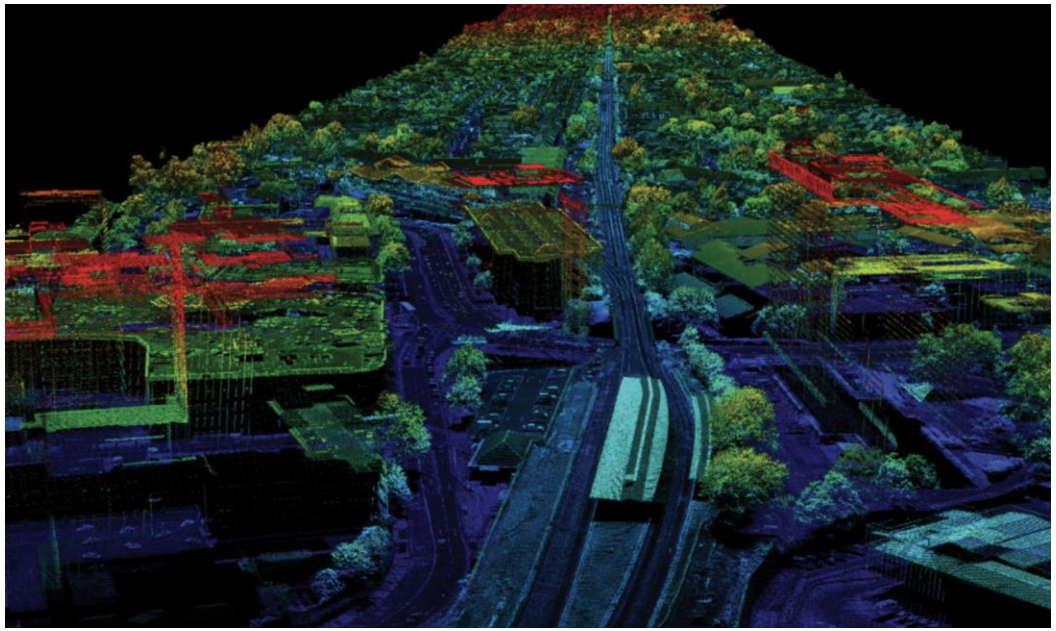


Recap: LiDAR-Visual Navigation

LiDAR-Visual Fusion Architecture

Block diagram:

•Sensors → Feature Extraction → Multi-Modal Fusion → Odometry → Mapping → Loop Closure → Global Optimization



Sensor Models

•Camera: projection, epipolar geometry

•LiDAR: SE(3) scan motion compensation

•IMU: Pre-integration

$$\mathbf{T}_{k+1} = \exp(\Delta\xi) \cdot \mathbf{T}_k$$

Recap: LiDAR Odometry

Goal: Estimate ego-motion by aligning LiDAR scans → build accurate trajectory & local map.

•Scan-to-scan / scan-to-map

Strategy	Description	Use Case
Scan-to-Scan	Align consecutive scans	Fast motion update, low latency
Scan-to-Map	Align current scan to local map	Higher accuracy, drift reduction

ICP Formulations

•**Point-to-Point ICP:** minimize $\|Rp + t - q\|^2$

•**Point-to-Plane ICP (preferred):** minimize $(n^T(Rp + t - q))^2$

→ Faster convergence using **surface normals**

Normal Constraints

•Normals estimated via local neighborhood

•Normal-consistency improves correspondence reliability

•Reject mismatches & dynamic points

Recap: Typical System 1 — LOAM

LOAM Feature Extraction

- Edge sharp / flat surfaces
- Curvature metric
- Downsample flat points

LOAM Odometry

Residuals:

- Edge-to-line
- Plane-to-plane

Optimization:

$$\min \sum r_{edge}^2 + \sum r_{plane}^2$$

LOAM Mapping & Global Optimization

- Register to local map
- Re-optimize trajectory
- Loop closure via ICP

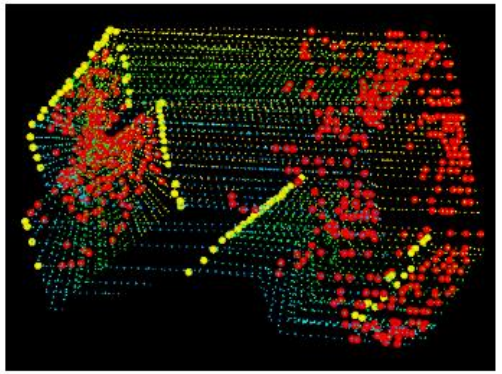


Fig. 5. An example of extracted edge points (yellow) and planar points (red) from lidar cloud taken in a corridor. Meanwhile, the lidar moves toward the wall on the left side of the figure at a speed of 0.5m/s, this results in motion distortion on the wall.

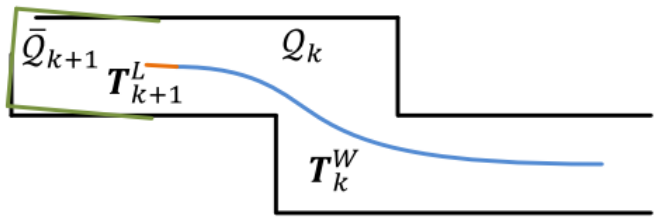


Fig. 8. Illustration of mapping process. The blue colored curve represents the lidar pose on the map, $\boldsymbol{T}_k^W$, generated by the mapping algorithm at sweep k. The orange color curve indicates the lidar motion during sweep $k+1$, $\boldsymbol{T}_{k+1}^L$, computed by the odometry algorithm. With $\boldsymbol{T}_k^W$ and $\boldsymbol{T}_{k+1}^L$, the undistorted point cloud published by the odometry algorithm is projected onto the map, denoted as $\bar{\mathcal{Q}}_{k+1}$ (the green colored line segments), and matched with the existing cloud on the map, $\mathcal{Q}_k$ (the black colored line segments).

Today's Lecture

Pipeline: Sensing → Perception → Planning → Control → Action

- Today's focus:
  - **Planning** → How an agent decides *where to go*
  - **Control** → How an agent *executes planned motion*

- Learning outcomes:
  - Understand core planning problems
  - Learn classical and modern planning methods
  - Understand how planners connect to perception and control layers

Trajectory Planning for Intelligent Vehicles

Definition: Finding a sequence of actions to achieve a goal given constraints and a model of the world.

Analogy: “Navigation GPS for robots.”

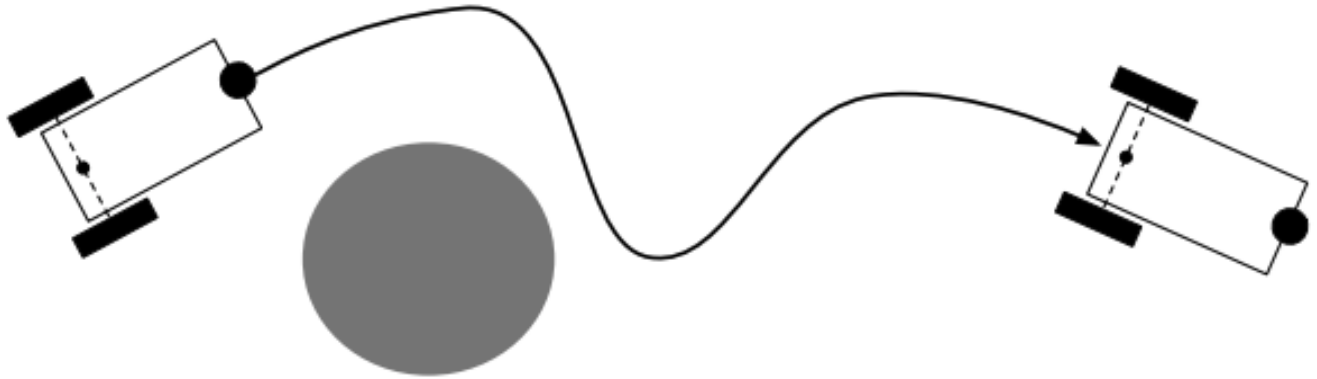
Planning vs. Decision-making vs. Control

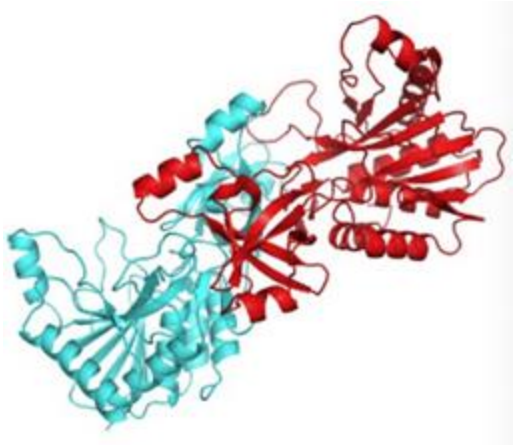
- Planning: *Strategic* (where to go)
- Control: *Tactical* (how to move)
- Decision-making: *High-level* (what to achieve)

Planning

More examples:

- Steering autonomous vehicles
- Controlling humanoid robot
- Surgery planning
- Protein folding
- …





Planning - Problem Formulation

Given:

1. An initial state of the world
2. A set of available actions, their requirements, and their effects
3. A goal state

Output:

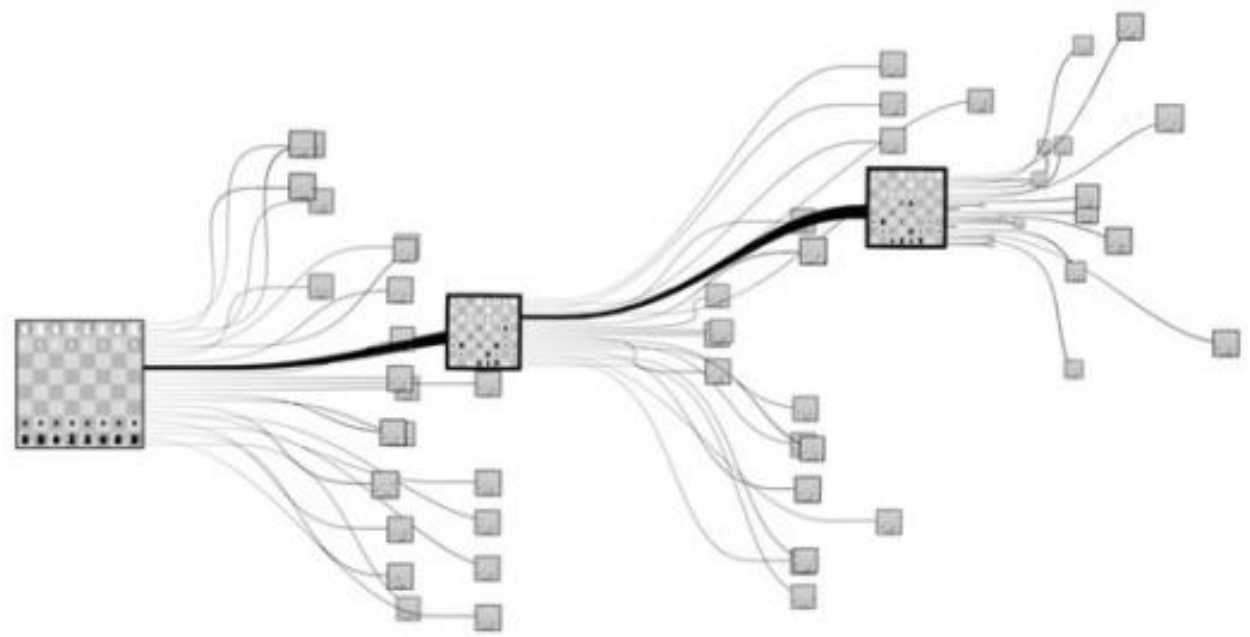
A valid sequence of actions that starts from **the initial state** and terminates at **the goal state**

History

- Planning is formally defined in the 1970s
- Development of exact, combinatorial solutions in the 1980s
- Development of sampling-based methods in the 1990s
- Deployment on real-time systems in the 2000s
- Current research: inclusion of differential and logical constraints, planning under uncertainty, parallel implementation, feedback plans and more

Planning For Chess

- Games
  - **Can we apply planning to play chess?**



Planning For Chess

Given:

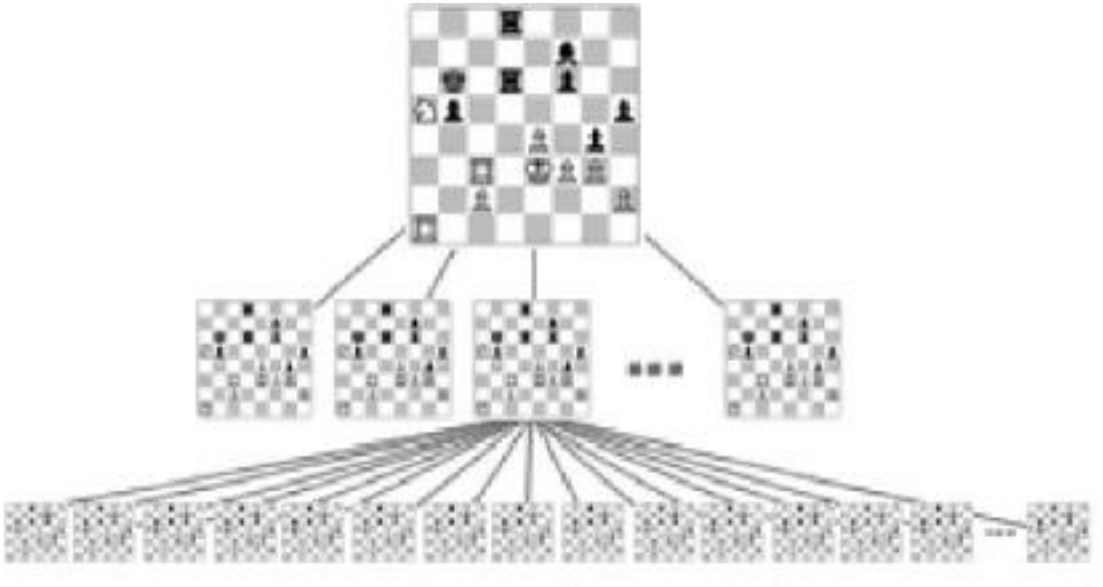
1. An initial state of the world **=> The chess board state at each turn**
2. A set of available actions, their requirements, and their effects **=> Chess moves**
3. A goal state **=> Checkmate conditions**

Compute:

A valid sequence of actions that starts from the initial state and terminates at the goal state
=> **Plan = sequence of chess moves**

Planning Via Tree Search

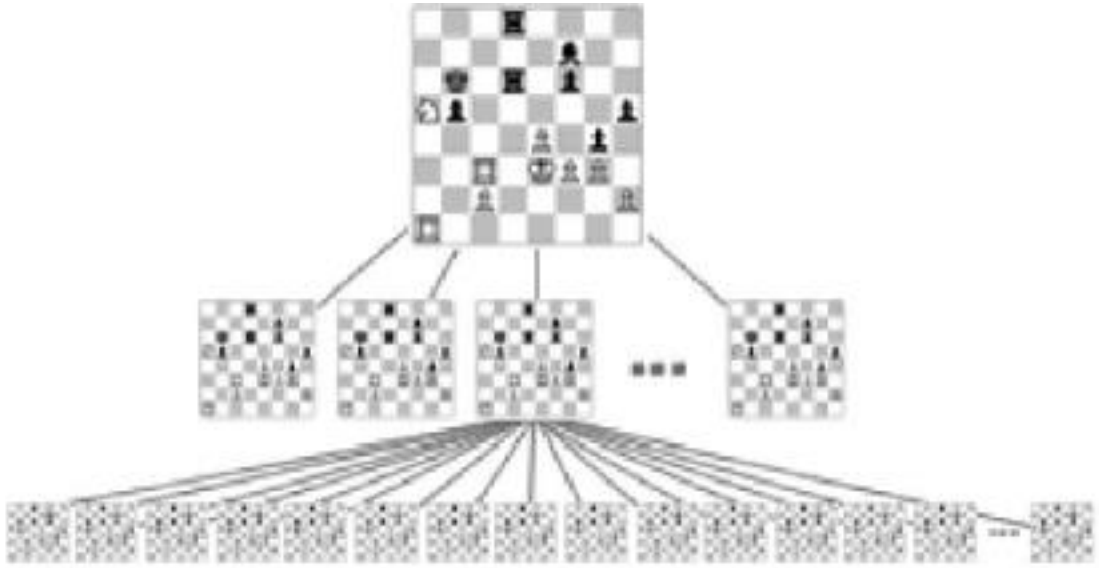
1. Enumerate all valid moves available to player
2. Enumerate all valid moves available to opponent
3. Repeat 1, 2 until the game ends
4. Pick action that had the highest likelihood of winning



Problem: Steps 1, 2 will enumerate far more states than computationally tractable!

Planning Via Tree Search

1. Enumerate all valid moves available to player
2. Enumerate all valid moves available to opponent
3. Repeat 1, 2 until the game ends **or up to d steps ahead**
4. Pick action that had the highest likelihood of winning, **using an estimated "value/chance of winning" of the state if the game did not end.**



Planning Via Tree Search

1997: IBM's Deep Blue defeats Garry Kasparov

Algorithm: Tree search with some clever strategies to prune the tree (alpha-beta pruning)

"Secret Sauce": Special chips designed to speed up enumeration and evaluation of chess moves

Planning Via Tree Search

Limitations of Playing Games Via Tree Search

Exponential Growth of Search Space:

Branching factor ↑ → nodes explode
Impossible to search all possibilities in complex games

Computationally Expensive:

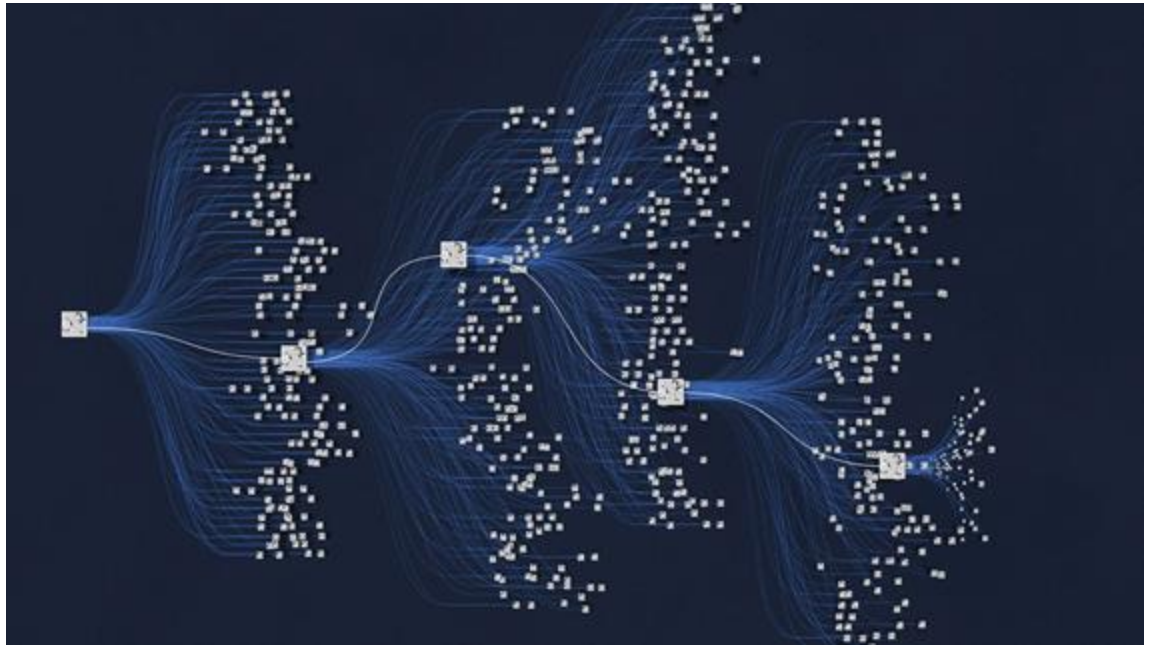
Deep search requires massive compute & memory
Hard to achieve real-time response

There are games harder than chess (e.g. Go) where steps 1,2 cannot be done even for a small number of steps of lookahead!

Planning Via Tree Search

AlphaGo: Tree Search + Policy + Value Neural Networks

1. Enumerate ~~all~~ **promising moves from policy neural network**
2. Enumerate ~~all~~ **promising moves** available to opponent (same network)
3. Repeat 1, 2 until the game ends **or up to d steps ahead**
4. Pick action that had the highest likelihood of winning, **using learned value neural network**



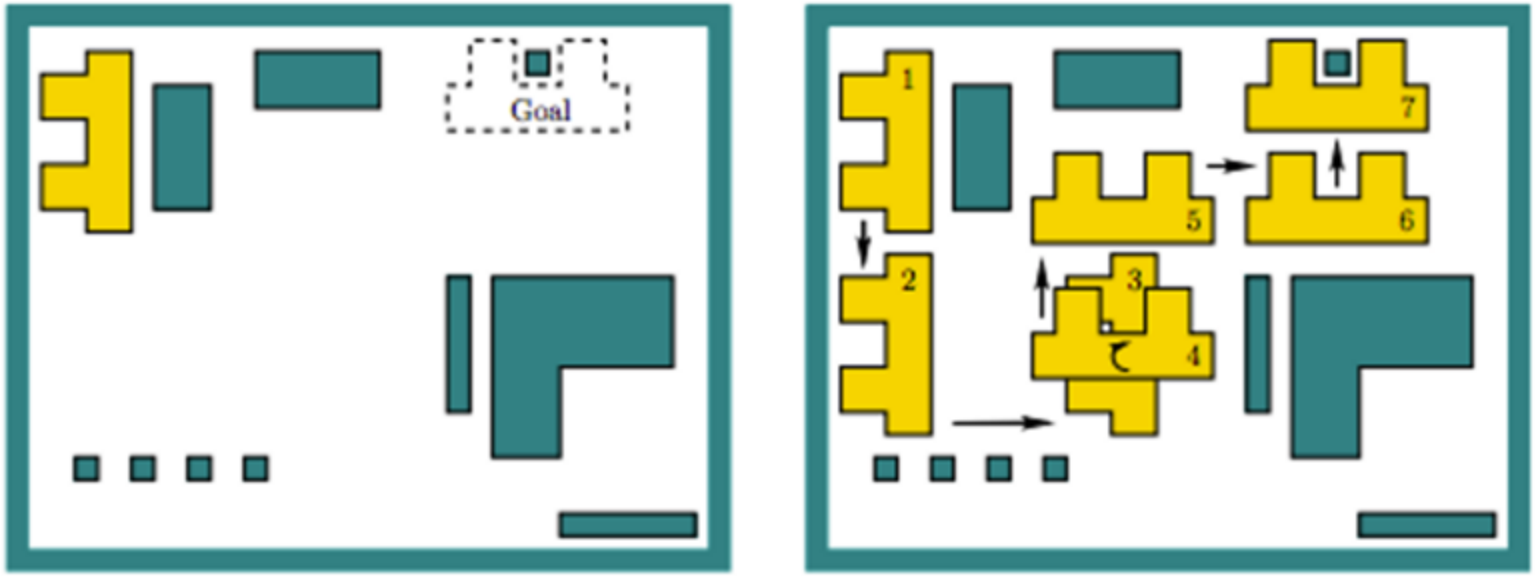
From Games to Robotics

- Discrete state space to continuous state action space
- Easy to simulate moves - no expensive physics / geometric computation
  - Simply place the piece on the chess board
- Rules of game already known - no notion of model uncertainty
  - We can compute the value/reward easily based on the game rules
  - In robotics, we need to simulate the robot or run the program on real robot to learn what will happen

Planning for Robotics

- Assume 2D workspace: $\mathcal{W} \subseteq \mathbb{R}^2$
- $\mathcal{O} \subset \mathcal{W}$ is the obstacle region with polygonal (straight-line) boundary
- Robot is a rigid polygon

Problem: given initial placement of robot, compute how to gradually move it into a desired goal placement so that it never touches the obstacle region



Planning for Robotics

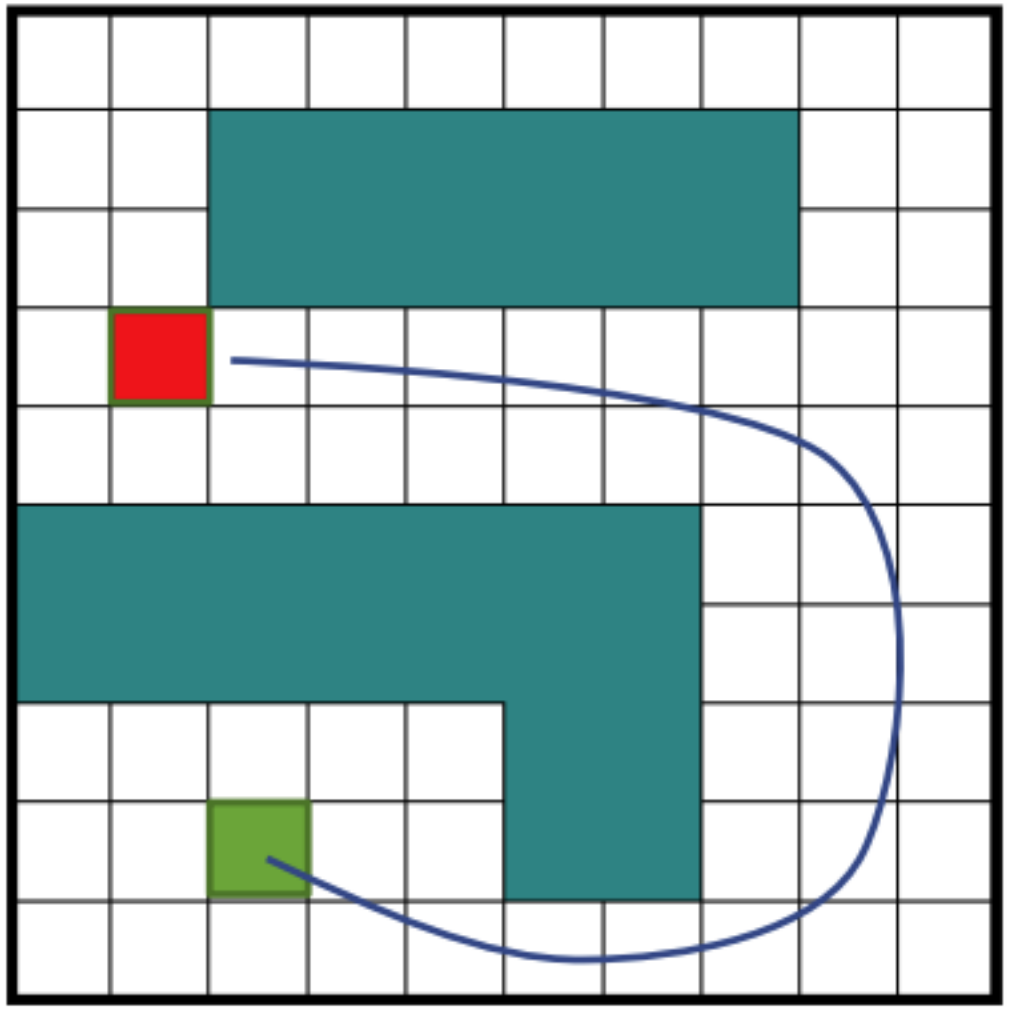
Popular Approaches

- **Potential fields** *[Rimon, Koditschek, '92]*: create forces on the robot that pull it toward the goal and push it away from obstacles
- **Grid-based planning** *[Stentz, '94]*: discretizes problem into grid and runs a graph-search algorithm (Dijkstra, A*, …)
- **Combinatorial planning** *[LaValle, '06]*: constructs structures in the configuration space (C-space) that completely capture all information needed for planning
- **Sampling-based planning** *[Kavraki et al, '96; LaValle, Kuffner, '06, etc.]*: uses collision detection algorithms to probe and incrementally search the C-space for a solution

Grid-based Approaches

- Discretize the continuous world into a grid
  - Each grid cell is either free or forbidden
    - a.k.a. Occupancy grid map
  - Robot moves between adjacent free cells
  - **Goal:** find sequence of free cells from start to goal

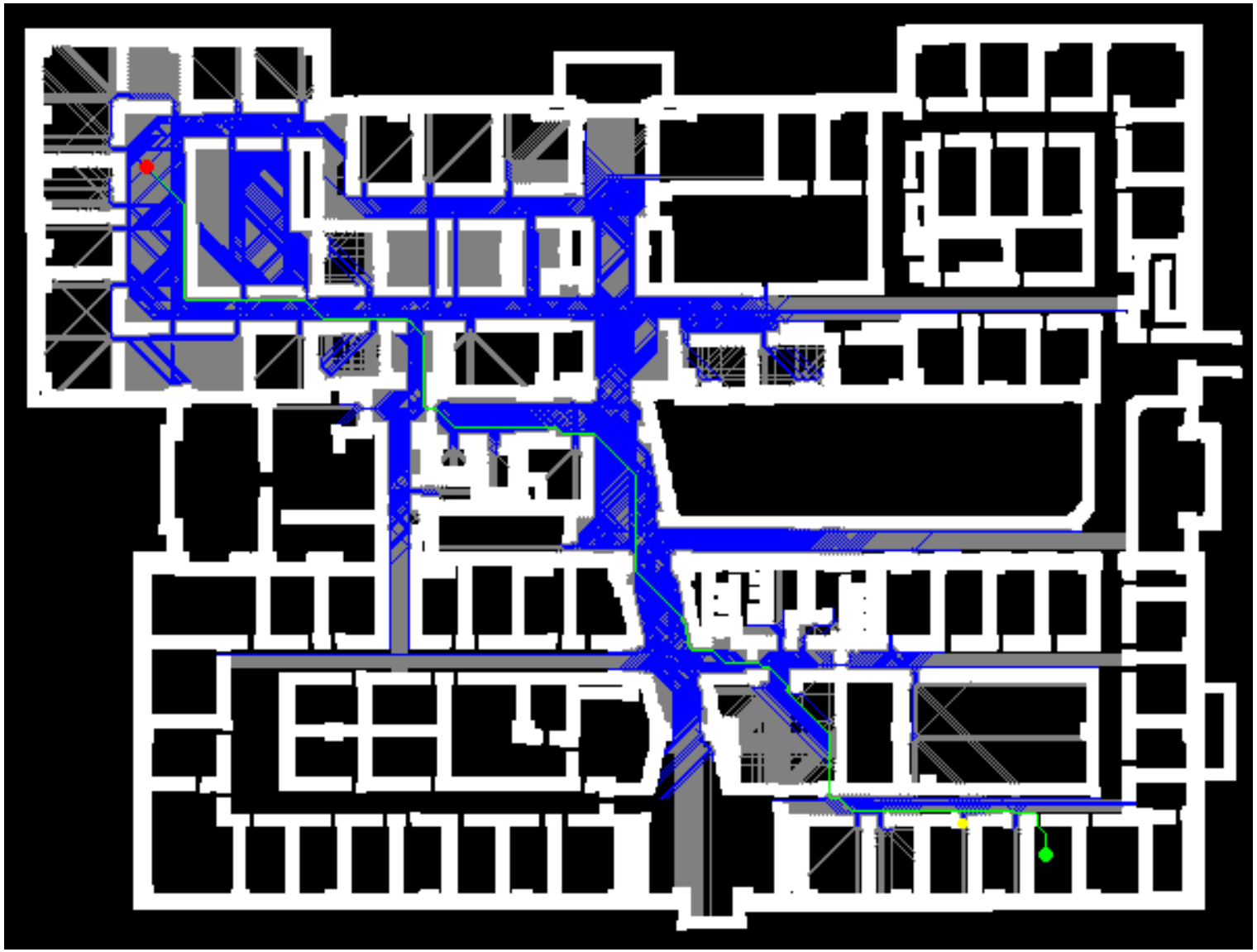
- Mathematically, this corresponds to pathfinding in a discrete **graph** G = (V, E)
  - Each vertex v represents a free cell
  - Edges connect adjacent grid cells
  - So let's look into graph search algorithm



Grid Planning for Robots

Given the occupancy grid map, start and end point.

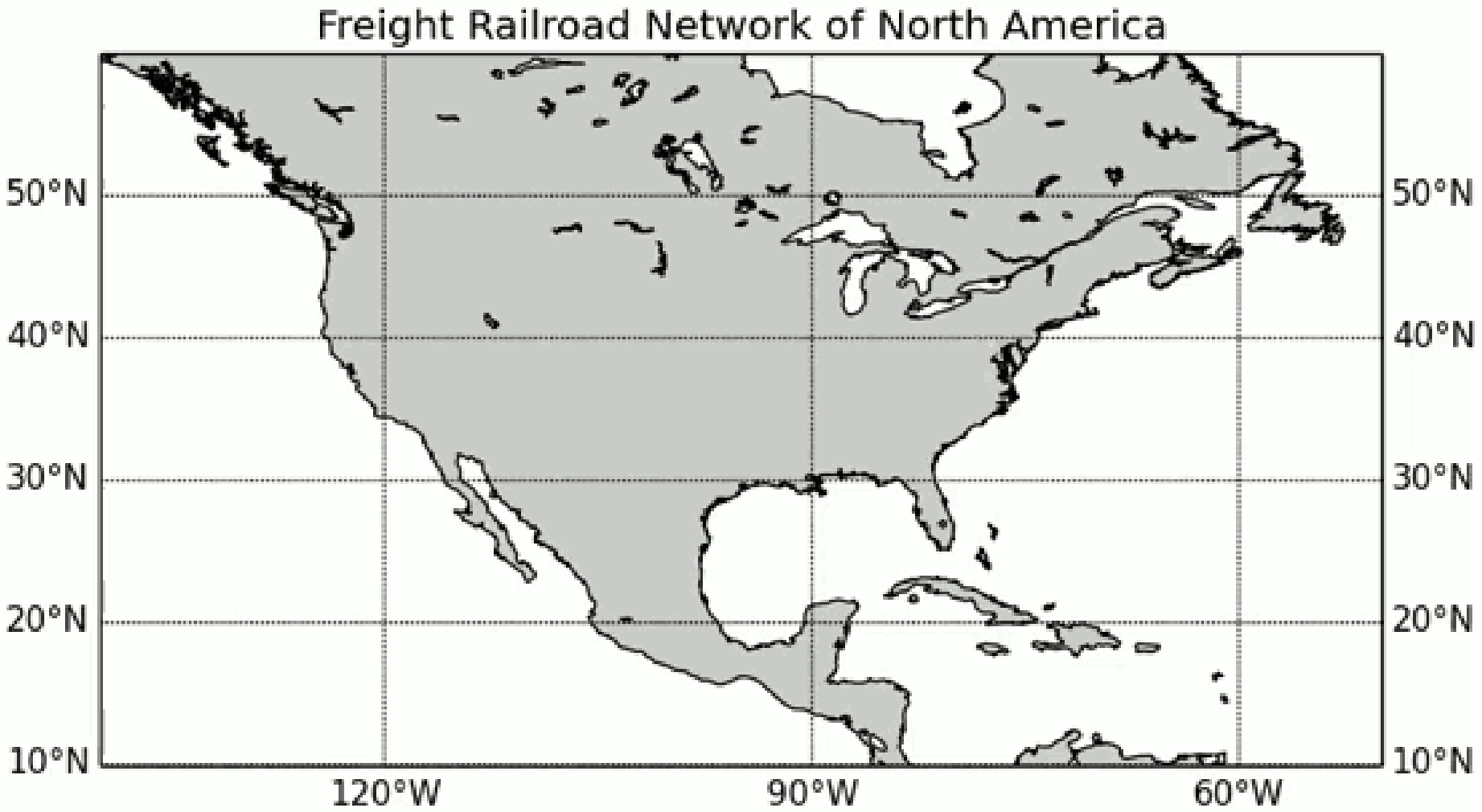
The blue are the traversed nodes and the green line is the optimal path



Real-world Planning Problem

A* to search a route between D.C. and LA

- Real-world application: finding an optimal path in the railroad network



Grid-based approaches

Summary

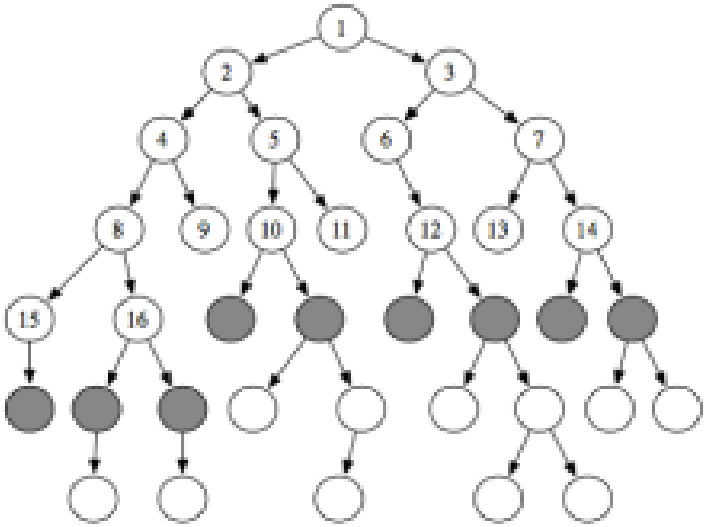
- Pros:
  - Simple and easy to use
  - Fast (for some problems)
- Cons
  - Limited to simple robots
    - Grid size is exponential in the number of DOFs

- Good tutorials for graph-search algorithm
  - https://github.com/npretto/pathfinding
  - For A*: https://www.redblobgames.com/pathfinding/a-star/introduction.html

Graph Search Algorithms

- Having determined problem, how to find "best" path?
  - **Breadth-first search**

- Maintain the path as a **queue** – First in/first first out
- Update cost for all edges up to current depth before proceeding to greater depth
- Edges all have the same cost
- Vertices explored in order of distance from start
- Best path found = path with fewest steps



Graph Search Algorithms

Dijkstra's Algorithm: BFS + Priority

Motivations:

- BFS (Breadth-First Search) works well only if all edge costs are equal (e.g., each move costs 1).
- What if edges have different travel costs? (e.g., rough terrain, slope, distance)
- Dijkstra's algorithm generalizes BFS to handle weighted graphs.

Intuition: BFS + Priority

•Like BFS, Dijkstra explores outward from the start.

•But instead of a **FIFO queue**, it uses a **priority queue (min-heap)**.

•**Priority = current *accumulated cost* from the start node.**

•The node with the **lowest total cost so far** is always expanded first.

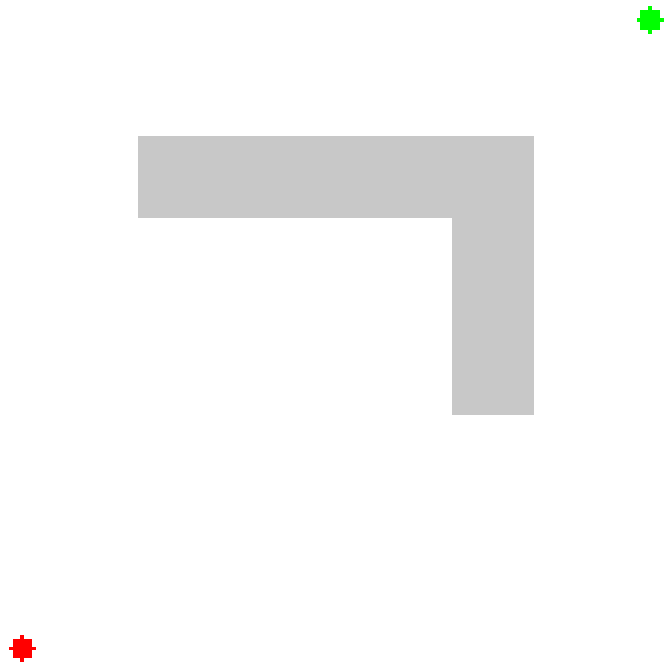
```
frontier = PriorityQueue()  // Note priority!
frontier.push(start, 0)     // 0 = cost to go
parent = {}, parent[start] = Null
cost = {}
cost[start] = 0             // Cost from start

while not frontier.empty():
  current = frontier.get()  // Get by priority!
  if current == goal:
    break
  for next in neighbors(current):
    new_cost = cost[current] + EdgeCost[current, next]
    if next not in cost or new_cost < cost[next]:
      cost[next] = new_cost  // Insertion or edit
      frontier.put(next, new_cost)
      parent[next] = current
```

Graph Search Algorithms

Dijkstra's Algorithm: BFS + Priority

- **Visualization**
  - Start from bottom left node to a goal node; set all other nodes to be unvisited with infinite cost
  - color representing distant to goal (greener = closer)



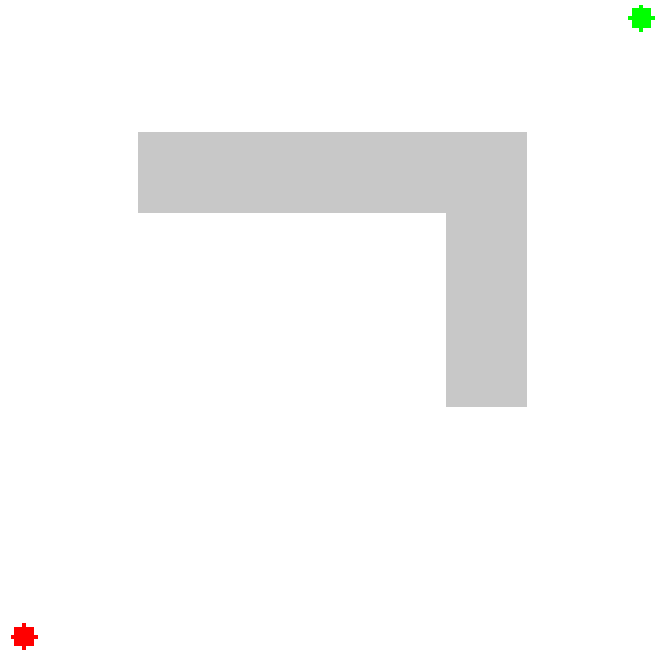
Algorithm Workflow:

1. Initialize all node costs = ∞, except start node = 0.
2. Push start node into a **priority queue** (keyed by cost).
3. While the queue is not empty:
   - Pop the node `u` with the lowest cost.
   - For each neighbor `v`:
     - If `cost[u] + w(u,v) < cost[v]`, then update:
       - `cost[v] = cost[u] + w(u,v)`
       - Push `(cost[v], v)` into the queue.
4. Repeat until all reachable nodes are processed.

Graph Search Algorithms

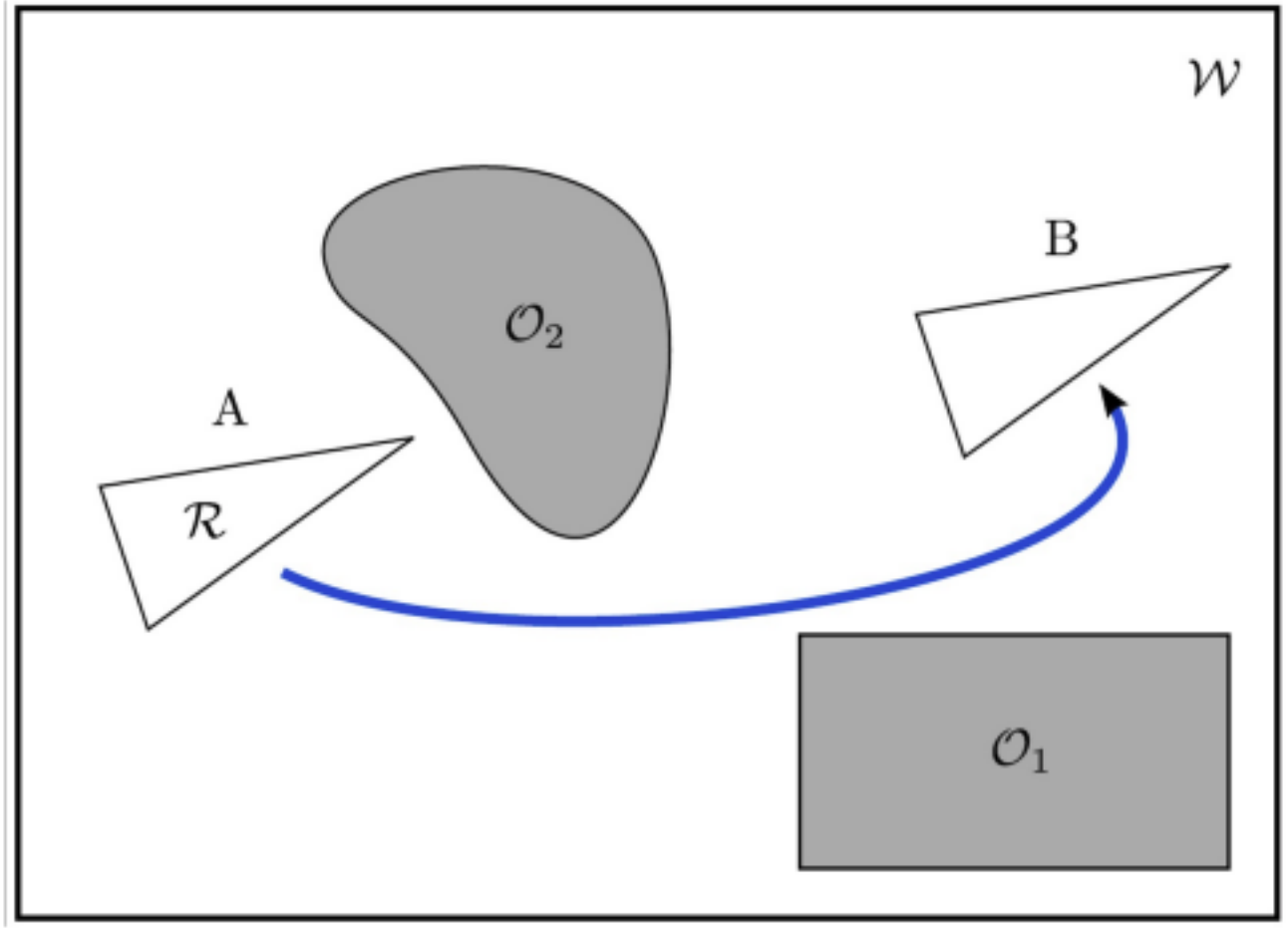
A*: Improving Dijkstra

- Dijkstra orders the path queue by optimal “cost-to-arrival”
- We can get faster results if order by “cost-to-arrival”+ (approximate) “cost-to-go”
  - So not just computing the cost of each node to the goal node but also a **heuristic for optimal cost-to-go to each node**
- In this way, fewer nodes will be placed in the frontier queue
- This modification still **guarantees** that the algorithm will terminate with a shortest path



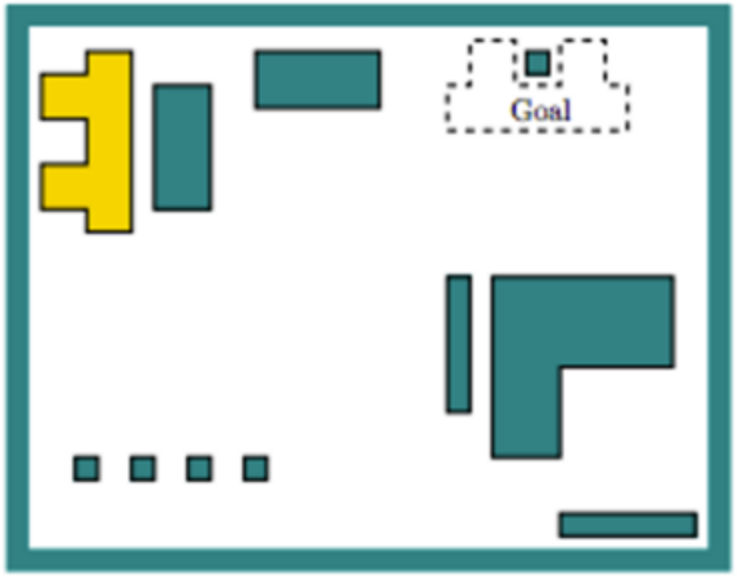
Back to continuous motion planning

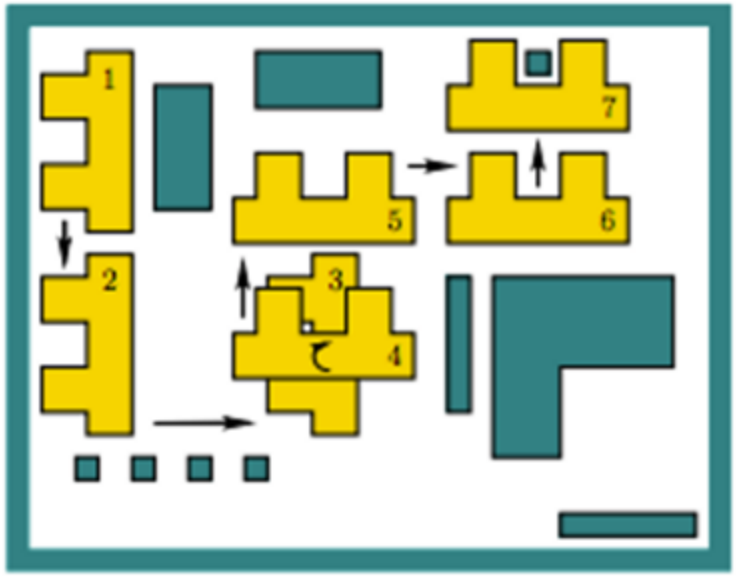
- A robot is a geometric entity operating in continuous space
- **Combinatorial techniques** for motion planning capture the structure of this continuous space
  - Particularly, the regions in which the robot is not in collision with obstacles
- Such approaches are typically complete
  - i.e., guaranteed to find a solution;
  - and sometimes even an optimal one



Planning for Robotics

- Assume 2D workspace: $\mathcal{W} \subseteq \mathbb{R}^2$
- $\mathcal{O} \subset \mathcal{W}$ is the obstacle region with polygonal boundary
- Robot is a rigid polygon
- **Problem:** given initial placement of robot, compute how to gradually move it into a desired goal placement so that it never touches the obstacle region



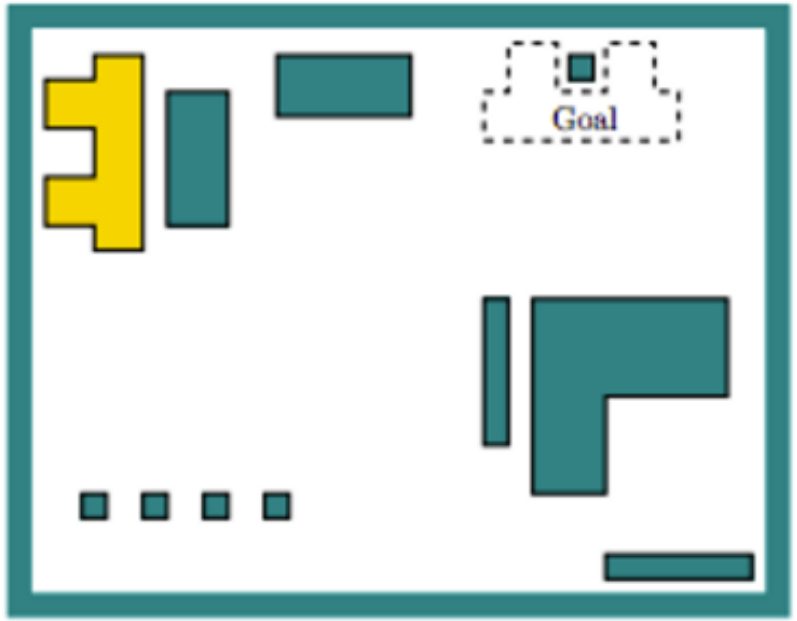


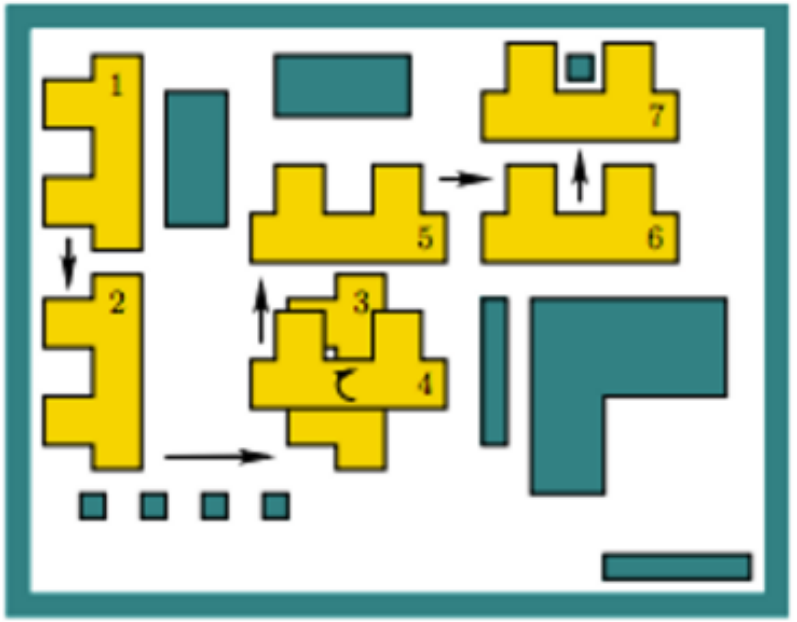
Step 1-3, translate
Step 4, rotate 90 degree clockwise,
Step 4-7, translate

Planning for Robotics

Simplest Setup

- Key point: motion planning problem described in the real-world, but it really lives in another space -- the **configuration space (C-space)** !



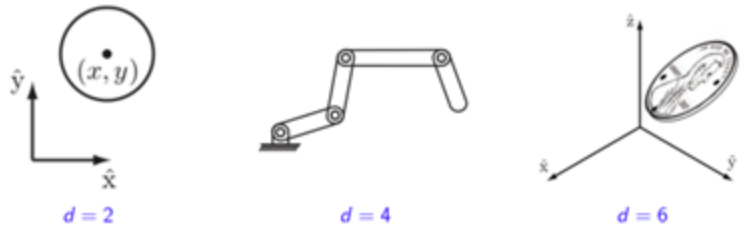


Planning for Robotics

Configuration space

- C- space: captures all degrees of freedom (all rigid body transformations)
- For example, le $\mathcal{R} \subset \mathbb{R}^2$ be a polygonal robot (e.g., a triangle)
- The robot can rotate by angl θ or transla $(x_t, y_t) \subset \mathbb{R}^2$
- Every combination $q = (x_t, y_t, \theta)$ yields a unique robot placement: configuration
- So, C- space is a subset of $\mathbb{R}^3$
- Note: $\theta \pm 2\pi$ yields equivalent rotations ⇒ C- space is: $\mathbb{R}^2 \times \mathcal{S}^1$
  - Also called SE(2): "**Special Euclidean group** of rigid body displacements in two-dimensions"
- Concept of C- space extends naturally to higher dimensions (e.g., robot linkages)

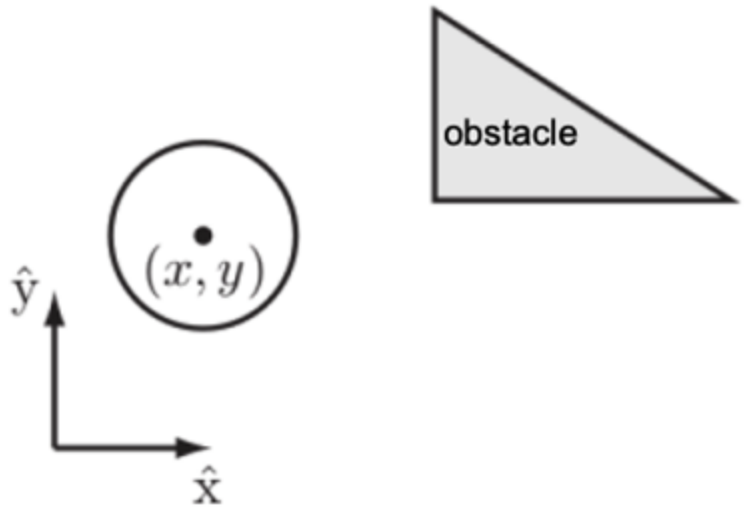
S^1: 1-dimensional sphere

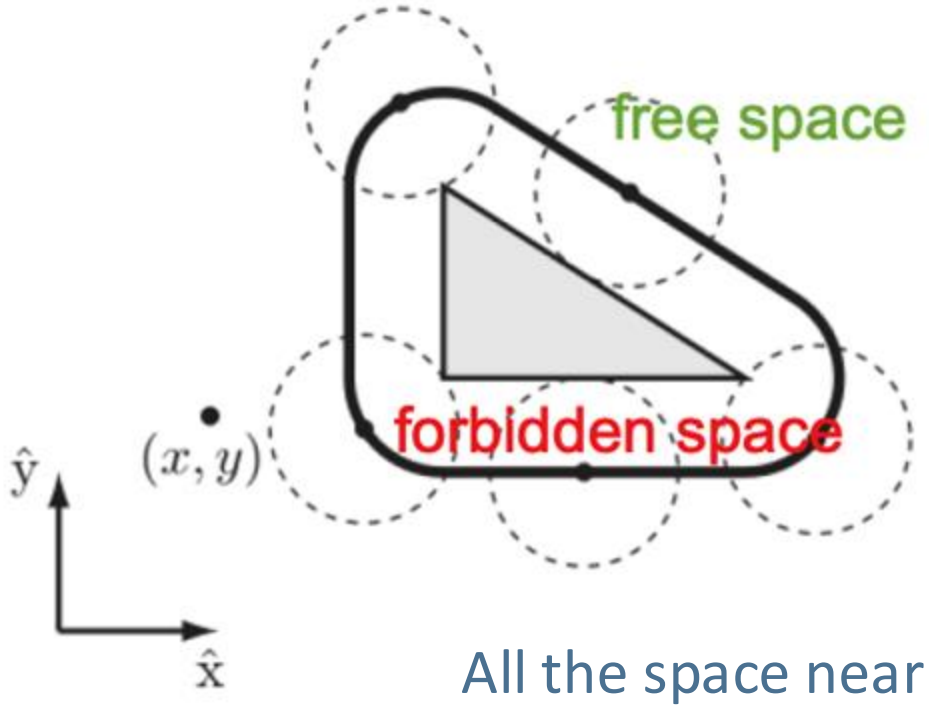


SE(3) for 3D space: Usually a 4x4 matrix to represent a point and a pose (3x3 rotation + 3x1 translation)

Configuration free space

- Consider a disk robot and the obstacle
  - The subset of all collision free configurations is the free space



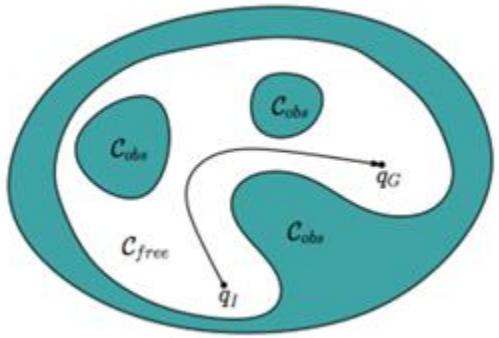


All the space near the obstacle within is obstacle space

Planning in C-space

- Let $R(q) \subset W$ denote the set of points in the world occupied by the robot when in configuration/state q
  - R(q) means the space the robot occupies in W
- Robot in collision $\Leftrightarrow$ $R(q) \cap O \neq \emptyset$
- Accordingly, free space is defined as: $C_{free} = \{q \in C | R(q) \cap O = \emptyset\}$
- Path planning problem in C-space: compute a continuous path:

$$\tau: [0,1] \rightarrow C_{free}, \text{with } \tau(0) = q_I \text{ and } \tau(1) = q_G$$



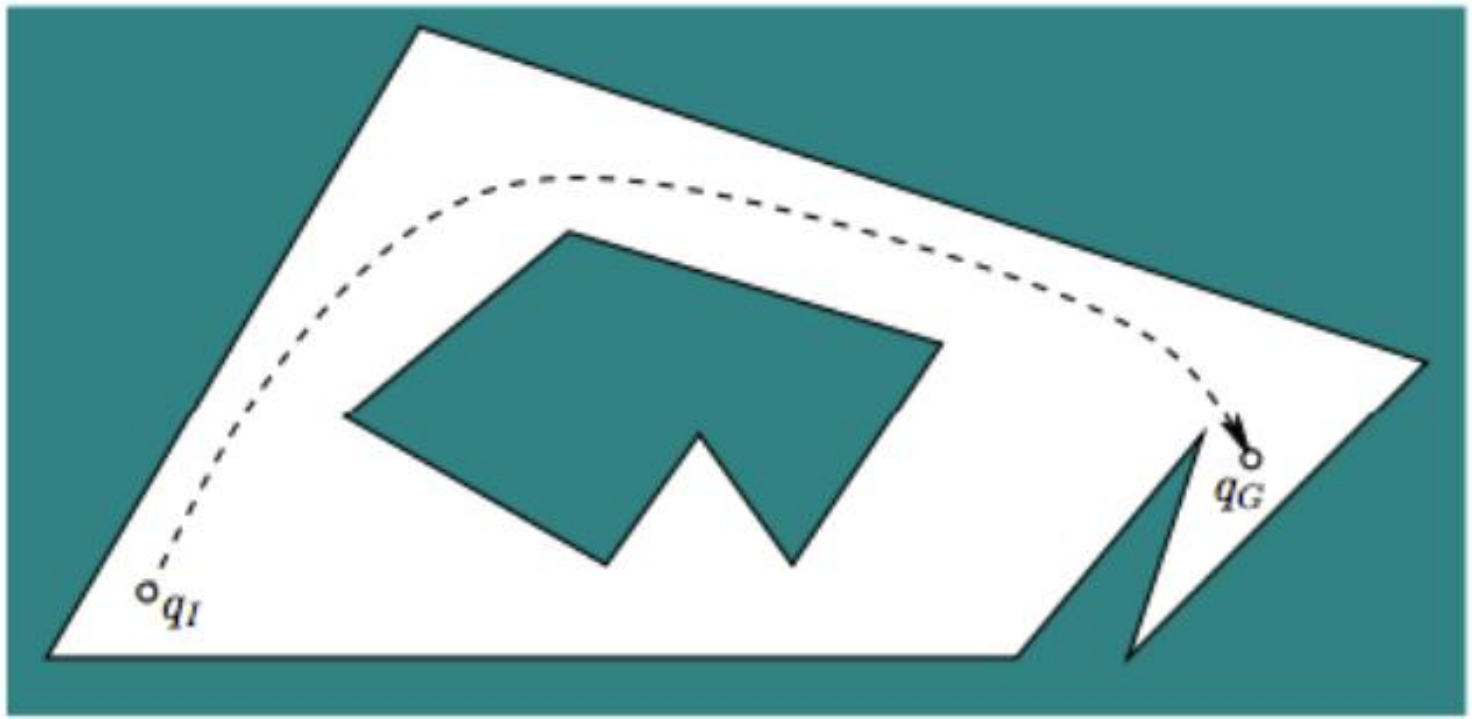
The [0, 1] represents a normalized parameter for the path, 0 being the start and 1 being the end

Motion planning algorithms

- Two main approaches to **continuous** motion planning:
  - **Combinatorial planning**: constructs structures in the C-space that **discretely and completely** capture all information needed to perform planning
  - **Sampling-based planning**: uses collision detection algorithms to probe and **incrementally search** the C-space for a solution, rather than completely characterizing all of the C_{free} structure

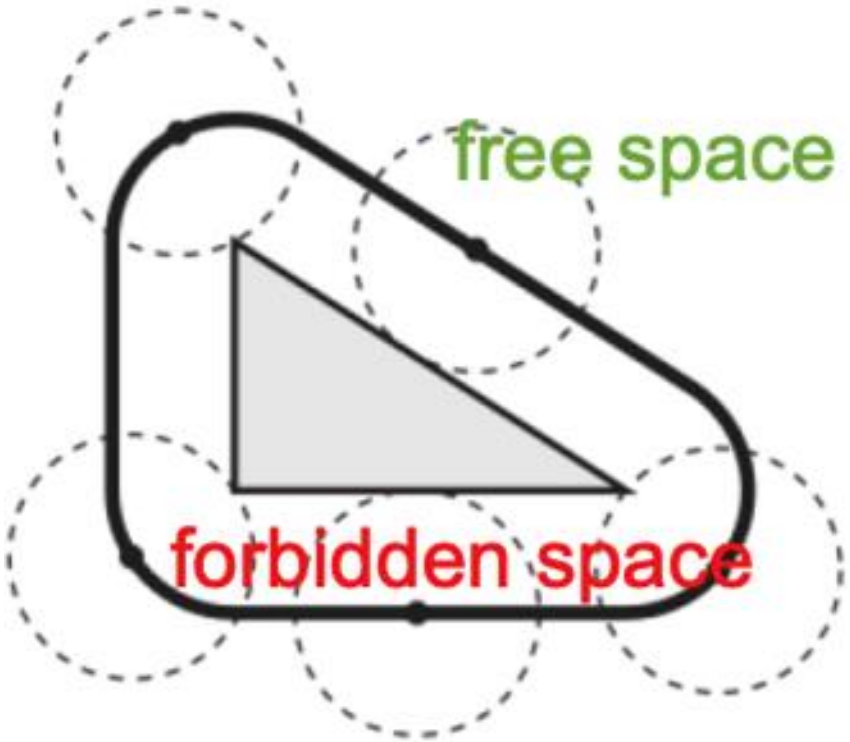
Combinatorial planning

- **Key idea:** compute a roadmap, which is a graph in which each vertex is a configuration in C_{free} and each edge is a path through C_{free} that connects a pair of vertices



Combinatorial planning - Free-space computation

- For the combinatorial approach, we need to compute the exact C_{free}
- The free space is not known in advance
- We need to compute this space given the ingredients
  - Robot representation, i.e., its shape (polygon, polyhedron, …)
  - Representation of obstacles
- To achieve this, we do the following:
  - Contract the robot into a point
  - In return, inflate (or stretch) obstacles by the shape of the robots



Combinatorial planning - summary

- These approaches are complete and even optimal in some cases
  - Do not discretize or approximate the problem
- Usually limited to small number of DOFs
  - Computationally intractable for many problems
- Difficult to implement; requires special software to reason about geometric data structures (CGAL*)
  - We will not get into details of this method

* de Berg et al., “Computational geometry: algorithms and applications”, 2008

Sampling-based motion planning

- Limitations of combinatorial approaches stimulated the development of sampling-based approaches
  - Abandon the idea of explicitly characterizing C_{free} and C_{obs}
  - Instead, capture the structure of C-space by random sampling
  - Use a black-box component (collision checker) to determine which random configurations lie in C_{free}
  - Use such a probing scheme to build a roadmap and then plan a path

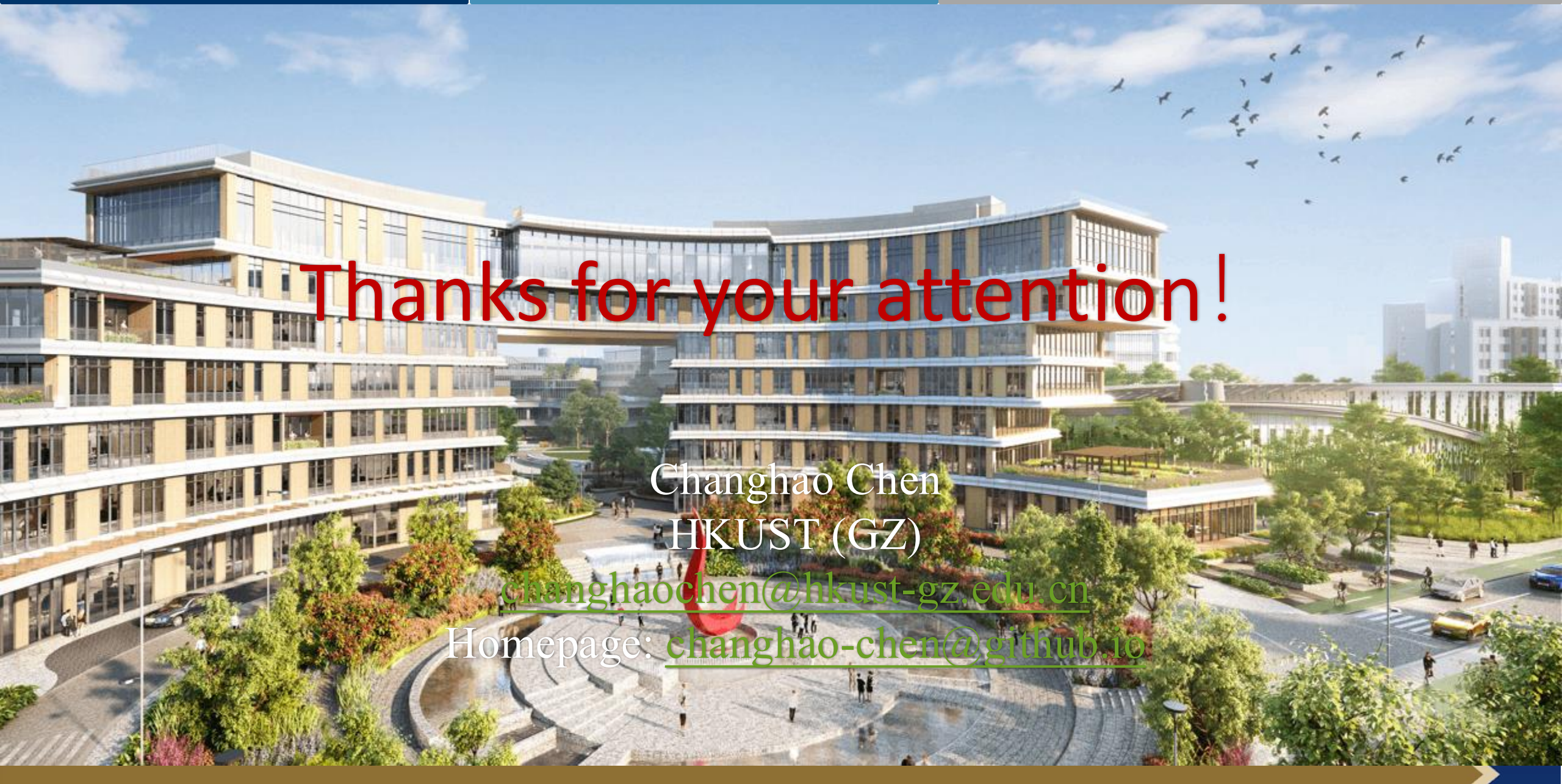
Reference: LaValle, S. M. Motion planning. 2011.

Sampling-based motion planning

- Pros:
  - Conceptually simple
  - Relatively-easy to implement
  - Flexible: one algorithm applies to a variety of robots and problems
  - Beyond the geometric case: can cope with complex differential constraints, uncertainty, etc.
- (Mild) cons:
  - Unclear how many samples should be generated to retrieve a solution
  - Cannot determine whether a solution does not exist
- Two major approaches
  - **Probabilistic Roadmap (PRM)**: graph-based
  - **Rapidly-exploring Random Trees (RRT)**: tree-based

In practise, A* is used extensively in many problems including indoor navigation with a known 2D map. And it is better than RRT in some cases[1]

[1] Zammit, Christian, and Erik-Jan Van Kampen. "Comparison between A* and RRT algorithms for 3D UAV path planning." Unmanned Systems 10.02 (2022): 129-146.



Thanks for your attention!

Changhao Chen
HKUST (GZ)
changhaochen@hkust-gz.edu.cn
Homepage: changhao-chen@github.io